

AetherBridge Voice Bridge — Technical Documentation

Team Aether_100 · Team ID NITS_100 · NIT Silchar · Problem Statement SIH26223 · Theme: Disaster Management · Category: Hardware
Mentor: Dr. Koushik Guha · September 2026

1. Purpose

AetherBridge is the voice subsystem of SentinelBridge. It carries two-way voice between an ordinary handheld radio and an IP network, so that a field responder using a standard walkie-talkie can talk to a district Emergency Operations Centre that is far outside radio range. The subsystem was built and validated on its own, separately from the flood-sensing tier, so that the radio-to-IP signal chain could be proven before integration.

2. The problem it solves

During a flood, responders in the field carry VHF handsets with a practical range of three to five kilometres. The district EOC may be tens of kilometres away. When the local cell tower is submerged or overloaded, the two cannot talk at all. Existing radio-over-IP gateways solve this, but they are imported, cost lakhs, and require agencies to replace handsets. AetherBridge extends the agency's existing base radio onto whatever IP link survives, using hardware that costs about ₹1,200 beyond the gateway computer, and requires no change to the handsets responders already carry.

3. Design principle — no new transmitter

The gateway contains no RF transmitter. It does two things only: it taps audio from the radio's accessory port rather than its antenna, and it closes the radio's own push-to-talk contact through an optocoupler — electrically identical to a human operator pressing the button. Every emission therefore comes from the radio's own transmitter. In deployment this is the responding agency's licensed VHF base station, so the gateway needs no spectrum allotment of its own. Bench testing was carried out on 446.0–446.2 MHz at 0.5 W, the band WPC has notified for licence-exempt personal mobile radio use.

4. Theory of operation

4.1 Field to EOC

A responder keys a handheld and speaks. The base radio receives the transmission and reproduces the audio on its accessory-port speaker output. That audio passes through a DC-blocking capacitor and a trimmer that scales the radio's speaker-level signal down to

microphone level, and into the USB sound card's microphone input.

The gateway computes a rolling RMS on each 10 ms frame. When the level exceeds a calibrated threshold, a software squelch gate opens and frames are forwarded over the IP link. The gate stays open for a short hang period after the level drops, so that natural pauses in speech do not chop the transmission into fragments. At the EOC, frames are played into the operator's headset and shown on a console as a live waveform and a timestamped log. Software squelch is necessary because the radio provides no carrier-detect or COS line on its accessory port. The gateway has no way of being told when the radio is receiving, so it infers it from the audio itself.

4.2 EOC to field

The EOC operator speaks into a headset. A voice-activity gate on the console sends frames over the IP link when speech is present. On arrival at the gateway, the software asserts GPIO17, which drives the optocoupler and closes the radio's PTT contact.

The gateway then waits 250 ms before sending any audio. This lead time lets the radio's power amplifier come up and the receiving handset's squelch open; without it, the first syllable of every sentence is lost. Audio is written to the sound card, through a fixed 100 k Ω resistor and a trimmer that attenuates line level down to the millivolt level an electret microphone input expects, and into the radio's microphone pin.

PTT is held for 350 ms after the last audio frame, then released. A hard 30-second timeout drops PTT unconditionally regardless of what the software believes, as a backstop against a stuck carrier.

4.3 Half-duplex interlock

A radio cannot receive while transmitting. While PTT is asserted, the gateway discards captured audio rather than forwarding it, and the EOC console suppresses its own microphone for a short period after receiving audio. Without these two interlocks the system forms an acoustic and electrical feedback loop the moment both ends are live.

5. The accessory port, and the fault it creates

The radio uses a Kenwood two-pin accessory port: a 3.5 mm and a 2.5 mm plug moulded together.

Plug and contact	Function	Harness wire
2.5 mm tip	Speaker +	Blue
2.5 mm sleeve	Speaker - / radio ground	Brown
3.5 mm ring	Microphone +	Red
3.5 mm sleeve	Microphone - and PTT sense	Green
3.5 mm tip	Supply voltage out — unused	—

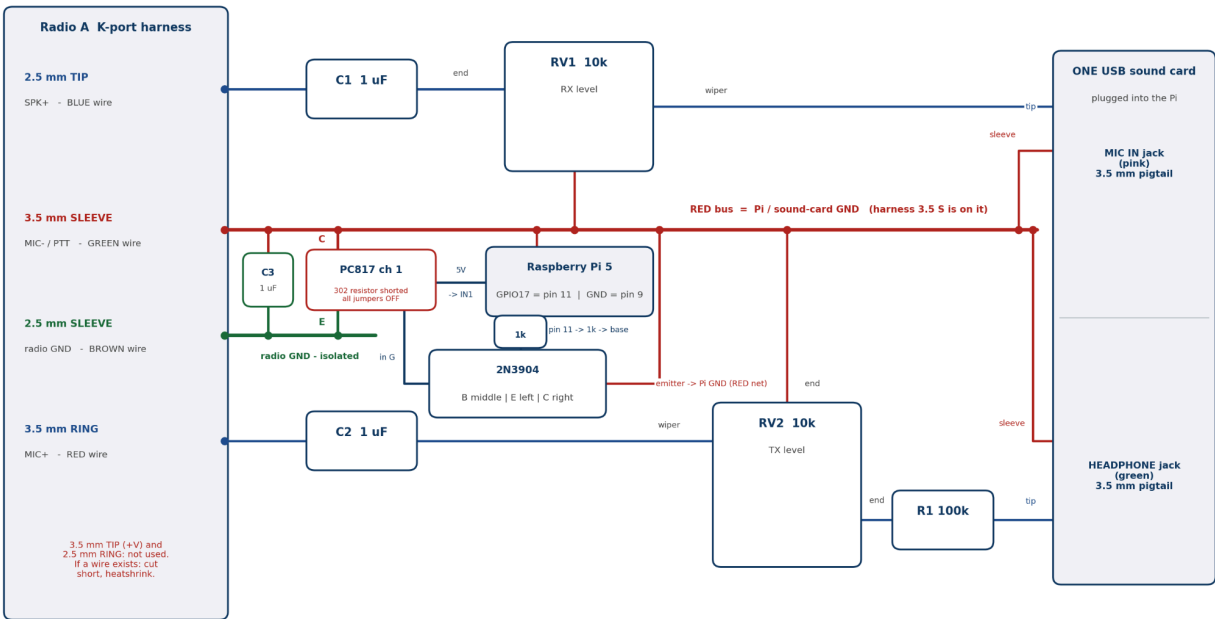
The microphone return and the PTT line are the same pin. The radio transmits whenever the

3.5 mm sleeve is connected to the 2.5 mm sleeve. Both contacts look like ground, and both are described as ground in most references, but they are separate nodes inside the radio. A USB sound card shares one internal ground between its microphone input and its headphone output. Wiring the radio's two returns to it in the obvious way therefore joins the two sleeves through the sound card and keys the transmitter continuously from the moment the harness is plugged in — a stuck carrier that overheats the power amplifier and occupies a shared channel. Preventing this is the central constraint of the interface design.

6. Interface circuit

AetherBridge - as-built wiring (no isolator, 2N3904 PTT drive)

Radio GND (green) reaches PI GND (red) ONLY through C3 (passes audio, blocks DC) and the optocoupler. Check: harness 3.5 S to 2.5 S must read OPEN.



As built: PC817 channel 1 has its 302 output resistor bridged and all four jumpers removed. The 2N3904 drives the LED from 5 V - measured 0.15 V when keyed.

Dots = joins. No lines cross. Blue = audio. Film caps are non-polarised. Pots: 'end' pins to the bus side start at minimum level.

6.1 Two ground nets

The board maintains two separate ground nets:

Net	Members
Pi ground	Harness green wire (3.5 mm sleeve, MIC-/PTT); optocoupler output collector; one end of each trimmer; both sound-card pigtail sleeves; one leg of the isolation capacitor; Pi ground pin
Radio ground	Harness brown wire (2.5 mm sleeve); optocoupler output emitter; the other leg of

Net	Members
	the isolation capacitor. Nothing else.

The two nets meet at exactly two points. The isolation capacitor passes audio between them while blocking DC, so no current can flow to key the radio. The optocoupler bridges them galvanically, and only when the software asserts PTT. Isolation is confirmed by measuring between the brown and green harness wires with the gateway powered down: the reading must be open circuit.

6.2 Connections

Path	Chain
Receive	Blue wire → C1 → RV1 (10 kΩ) → microphone pigtail tip. RV1's other end on Pi ground.
Transmit	Headphone pigtail tip → R1 (100 kΩ) → RV2 (10 kΩ) → C2 → red wire. RV2's other end on Pi ground.
Isolation	C3 between the brown wire and Pi ground.
PTT drive	Pi pin 11 (GPIO17) → 1 kΩ → 2N3904 base; emitter → Pi pin 9; collector → optocoupler input ground; Pi pin 2 (5 V) → optocoupler input.
PTT output	Optocoupler collector → green wire; emitter → brown wire.

6.3 Why a transistor drives the optocoupler

Driving the optocoupler's LED directly from a 3.3 V GPIO pin produced insufficient current: the output transistor only pulled the PTT line down to 2.44 V, far above the threshold the radio needs, and it would not have keyed reliably.

Two changes fixed it. A 2N3904 now switches the LED against the 5 V rail, roughly doubling the LED current; and a 3 kΩ resistor the module carries in series with its output was bridged, since it dominated the on-state resistance. The arrangement is also fail-safe: with the software not running, the GPIO pin sits low, the transistor is off, and the radio cannot key.

Measured result: 0.15 V across the switch when keyed — a properly saturated closure.

6.4 Level setting

Both directions need heavy, adjustable attenuation, and the two are asymmetric. On receive, a radio speaker output of several hundred millivolts must be reduced to the tens of millivolts a microphone input expects. On transmit, a line-level output of around a volt must be reduced to roughly 10 millivolts for an electret microphone input. Over-driving the microphone input is the classic failure in this kind of build: it produces distorted, splattered audio that sounds like

interference rather than clipping. Twenty-five-turn trimmers are used so the usable range is not compressed into a fraction of a single turn.

7. Software

Element	Implementation
Platform	Raspberry Pi OS 64-bit (Debian 13), Raspberry Pi 5
Audio	Python, 48 kHz mono, 10 ms frames (480 samples). Frames are sized to fit a single network packet without fragmentation.
Squelch	Rolling RMS against a calibrated threshold, with hang time
PTT	GPIO17, with 250 ms lead, 350 ms tail, and a 30-second hard timeout
Jitter buffer	Bounded queue; oldest frames dropped beyond the bound so latency cannot accumulate through clock drift between the two sound cards
Transport	Voice frames over UDP, carried across two separate networks by an encrypted peer-to-peer overlay. No server sits in the voice path.
Resilience	Both ends survive loss of network without restarting; the bridge resumes automatically when connectivity returns.
EOC console	Live waveform, link state, timestamped transmission log, optional per-transmission WAV recording

8. Verification

The build was validated in a fixed sequence, each stage gated on a measurement, so that no fault could reach the radio.

Stage	Test	Result
1	Optocoupler input and output grounds isolated	Open circuit — confirmed
2	Board isolation: brown to green harness wire,	Open circuit — confirmed

Stage	Test	Result
	unpowered	
3	Idle on the radio, gateway powered, for 60 seconds	No transmission — confirmed
4	PTT keying under software control	Radio keys and releases on command; carrier heard at the second radio
5	Receive level, idle versus speech	Measurement pending
6	Transmit audio quality	Clean 1 kHz tone received at the second radio
7	Network round-trip time between the two networks	9–17 ms
8	End-to-end voice latency, median of 10 trials	Measurement pending

9. Demonstrated behaviour

- Voice from a field handheld arriving at an EOC console on a different network
- Voice from the EOC transmitted back over RF, with the gateway keying the radio unassisted
- Unmodified handsets at both ends — no new hardware, no retraining for responders
- Graceful degradation: with the IP link removed the radios continue to work locally, and the bridge resumes by itself when the link returns

10. Scope — what this prototype does not cover

Stated plainly so the boundary is unambiguous.

- The FloodSense sensing tier is not built: no deployed leaf nodes, no LoRa link, no on-hardware inference. The operations console runs the project's decision logic against synthetic telemetry.
- Bench testing used UHF licence-exempt handsets rather than an agency VHF base radio. The accessory tap, PTT relay and audio chain are identical in both cases; only the band differs.
- The receive-only SDR monitoring branch is part of the architecture but not part of this build.
- The cellular and satellite uplink paths remain design claims. The demonstrated IP link runs over ordinary internet connections.
- The prototype is a bench build. Weatherproofing and solar power are deployment claims, not demonstrated ones.

11. Cost

Interface hardware, excluding the gateway computer and radios: approximately ₹1,200. The design uses only commodity components available from Indian suppliers, with no vendor lock-in.

12. Repository

Circuit diagram, gateway software, EOC console and build procedure: github.com/jayInnovates
— open-source release planned.